

Assignment_2_cwood19

Chris Wood

2026-06-04

1: Based on the k-NN results of the nearest neighboring value, the suggested customer would decline the loan option. The majority of loan options are declined, so this appears to be a very feasible outcome by random chance.

2. K controls the bias:variance trade-off. Making a small K offers flexibility, low bias, and high variance, which leads to overfitting. Conversely, a large K will run higher bias and lower variance to push underfitting. When a dataset is imbalanced (480/5000), a large K will drown out the minority.
3. Show confusion matrix
4. Best K is still determined to be K=1 at 0.9635 and results do not change, still showing the loan option being declined.
5. The difference shown at K=1 is that accuracy devolves from Train to Validate to Test. Part of this is likely due to a reduced data set. The Train data exhibits no mismatches for prediction to result, but even though there are less total values in the Test set, it yields the highest mismatched predictions:results. Because the best bias is such a low value, the model is working with extreme low bias and class imbalance is still high.

```
# Load packages
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(fastDummies)
```

```
## Warning: package 'fastDummies' was built under R version 4.5.3
```

```
library(class)
```

```
## Warning: package 'class' was built under R version 4.5.3
```

```
# Load dataset
```

```
UniversalBank <- read.csv("UniversalBank.csv")
```

```
# Remove unused variables
```

```
UniversalBank <- UniversalBank[, !(names(UniversalBank) %in% c("ID", "ZIP.Code"))]
```

```

# Handle extra Education variables with dummies
UniversalBank <- dummy_cols(
  UniversalBank,
  select_columns = "Education",
  remove_first_dummy = FALSE,
  remove_selected_columns = TRUE
)

# Set reproducibility
set.seed(123)

# Train the model for 60% / p = .6; y = target variable
train_index <- createDataPartition(
  UniversalBank$Personal.Loan,
  p = 0.60,
  list = FALSE
)

train_data <- UniversalBank[train_index, ]
valid_data <- UniversalBank[-train_index, ]

# Normalize / Scale
predictor_names <- setdiff(names(train_data), "Personal.Loan")

preproc <- preProcess(
  train_data[, predictor_names],
  method = c("center", "scale")
)

train_x <- predict(preproc, train_data[, predictor_names])
valid_x <- predict(preproc, valid_data[, predictor_names])

##### 1: Create Sample Record #####
new_customer <- data.frame(
  Age = 40,
  Experience = 10,
  Income = 84,
  Family = 2,
  CCAvg = 2,
  Mortgage = 0,
  Securities.Account = 0,
  CD.Account = 0,
  Online = 1,
  CreditCard = 1,
  Education_1 = 0,
  Education_2 = 1,
  Education_3 = 0
)

new_customer <- predict(preproc, new_customer)

pred <- knn(
  train = train_x,

```

```

test = new_customer,
# Define Success factor of 0=declined and 1 = accepted
cl = factor(train_data$Personal.Loan),
# Determine nearest neighbor accepting or declining
k = 1,
prob = TRUE
)

# Convert results to table
result <- data.frame(
  Prediction = pred,
  Label = ifelse(pred == 1, "Accept", "Decline"),
  Confidence = round(attr(pred, "prob") * 100, 2)
)

print(result)

```

```

## Prediction Label Confidence
## 1          0 Decline      100

```

```

##### 3: Confusion matrix of K #####
# Find the best K
k_values <- seq(1, 25, 2)
accuracy <- rep(0, length(k_values))

for (i in seq_along(k_values)) {
  pred <- knn(
    train = train_x,
    test = valid_x,
    cl = train_data$Personal.Loan,
    k = k_values[i]
  )

  accuracy[i] <- mean(pred == valid_data$Personal.Loan)
}

results <- data.frame(k = k_values, accuracy = accuracy)
results

```

```

##      k accuracy
## 1    1  0.9635
## 2    3  0.9585
## 3    5  0.9545
## 4    7  0.9545
## 5    9  0.9495
## 6   11  0.9465
## 7   13  0.9465
## 8   15  0.9435
## 9   17  0.9435
## 10  19  0.9420
## 11  21  0.9400
## 12  23  0.9400
## 13  25  0.9395

```

```
best_k <- results$k[which.max(results$accuracy)]
best_k
```

```
## [1] 1
```

```
# Predict again using the best K
```

```
best_pred <- knn(
  train = train_x,
  test = valid_x,
  cl = train_data$Personal.Loan,
  k = best_k
)
```

```
# Confusion Matrix
```

```
conf_matrix <- table(
  Predicted = best_pred,
  Actual = valid_data$Personal.Loan
)
```

```
# Confusion Matrix table visual
```

```
cm <- table(
  Predicted = best_pred,
  Actual = valid_data$Personal.Loan
)
```

```
cm_df <- as.data.frame(cm)
```

```
cm_df$Predicted <- ifelse(cm_df$Predicted == 1, "Accept", "Decline")
cm_df$Actual <- ifelse(cm_df$Actual == 1, "Accept", "Decline")
```

```
names(cm_df)[3] <- "Count"
```

```
cm_df
```

```
##   Predicted Actual Count
## 1  Decline Decline  1776
## 2   Accept Decline    22
## 3  Decline  Accept    51
## 4   Accept  Accept   151
```

```
##### 4: Classify previous customer with best K #####
```

```
new_customer_4 <- predict(preproc, new_customer)
```

```
pred_4 <- knn(
  train = train_x,
  test = new_customer_4,
  # Define Success factor of 0=declined and 1 = accepted
  cl = train_data$Personal.Loan,
  # Determine nearest neighbor accepting or declining
  k = best_k,
  prob = TRUE
)
```

```

# Convert results to table
result_4 <- data.frame(
  Prediction = pred_4,
  Label = ifelse(pred_4 == 1, "Accept", "Decline"),
  Confidence = round(attr(pred_4, "prob") * 100, 2)
)

print(result_4)

##      Prediction   Label Confidence
## 1             0 Decline         100

##### 5: Repartition with best K 50:30:20 #####
n <- nrow(UniversalBank)
# Split into 50/50
train_index <- createDataPartition(UniversalBank$Personal.Loan, p = 0.50, list = FALSE)
train_data <- UniversalBank[train_index, ]

temp_data <- UniversalBank[-train_index, ]
# Split 50 into 60/40
valid_index <- createDataPartition(temp_data$Personal.Loan, p = 0.60, list = FALSE)
valid_data <- temp_data[valid_index, ]
test_data <- temp_data[-valid_index, ]

# Make predictions
predictor_names <- setdiff(names(train_data), "Personal.Loan")

train_x <- train_data[, predictor_names]
valid_x <- valid_data[, predictor_names]
test_x <- test_data[, predictor_names]

# Scale for training only
preproc <- preProcess(train_x, method = c("center", "scale"))

train_x <- predict(preproc, train_x)
valid_x <- predict(preproc, valid_x)
test_x <- predict(preproc, test_x)

# Fit best K
train_pred <- knn(train_x, train_x, train_data$Personal.Loan, k = best_k)
valid_pred <- knn(train_x, valid_x, train_data$Personal.Loan, k = best_k)
test_pred <- knn(train_x, test_x, train_data$Personal.Loan, k = best_k)

# New confusion matrix
# Train
cm_train <- table(Predicted = train_pred, Actual = train_data$Personal.Loan)

cm_train_df <- as.data.frame(cm_train)
names(cm_train_df)[3] <- "Count"

cm_train_df

##      Predicted Actual Count

```

```
## 1      0      0 2249
## 2      1      0    0
## 3      0      1    0
## 4      1      1  251
```

```
# Validate
cm_train <- table(Predicted = valid_pred, Actual = valid_data$Personal.Loan)

cm_train_df <- as.data.frame(cm_train)
names(cm_train_df)[3] <- "Count"

cm_train_df
```

```
##   Predicted Actual Count
## 1          0      0  1347
## 2          1      0    16
## 3          0      1    43
## 4          1      1    94
```

```
# Test
cm_train <- table(Predicted = test_pred, Actual = test_data$Personal.Loan)

cm_train_df <- as.data.frame(cm_train)
names(cm_train_df)[3] <- "Count"

cm_train_df
```

```
##   Predicted Actual Count
## 1          0      0   889
## 2          1      0    19
## 3          0      1    28
## 4          1      1    64
```

```
# Nice table
data.frame(
  Set = c("Train", "Validate", "Test"),
  Accuracy = c(
    mean(train_pred == train_data$Personal.Loan),
    mean(valid_pred == valid_data$Personal.Loan),
    mean(test_pred == test_data$Personal.Loan)
  )
)
```

```
##           Set Accuracy
## 1      Train 1.0000000
## 2  Validate 0.9606667
## 3       Test 0.9530000
```